Image Generation & Multi-modal models

- By Dr. Anand Jayaraman

© 2025 TekFrameworks

Agenda

- Multilingual models
- Image Generation – Very brief intro to diffusion models
- CLIP
- Vision Transformers
- Multimodal models
- Architecture & Enterprise patterns
- Evaluation

© 2025 TekFrameworks

Beyond English

- Problem: Most training data is English. Most of the world is not.
- The old way (Pipeline):
  - *Translate --> Process (in English) --> Translate back*.
  - Latency high, error propagation high
- The New way (Native Multilingualism): One model, many languages. The model “thinks” in concepts, not specific English words.
- The Goal: Cross-lingual generalization (Train on English, infer on Tamil)

© 2025 TekFrameworks

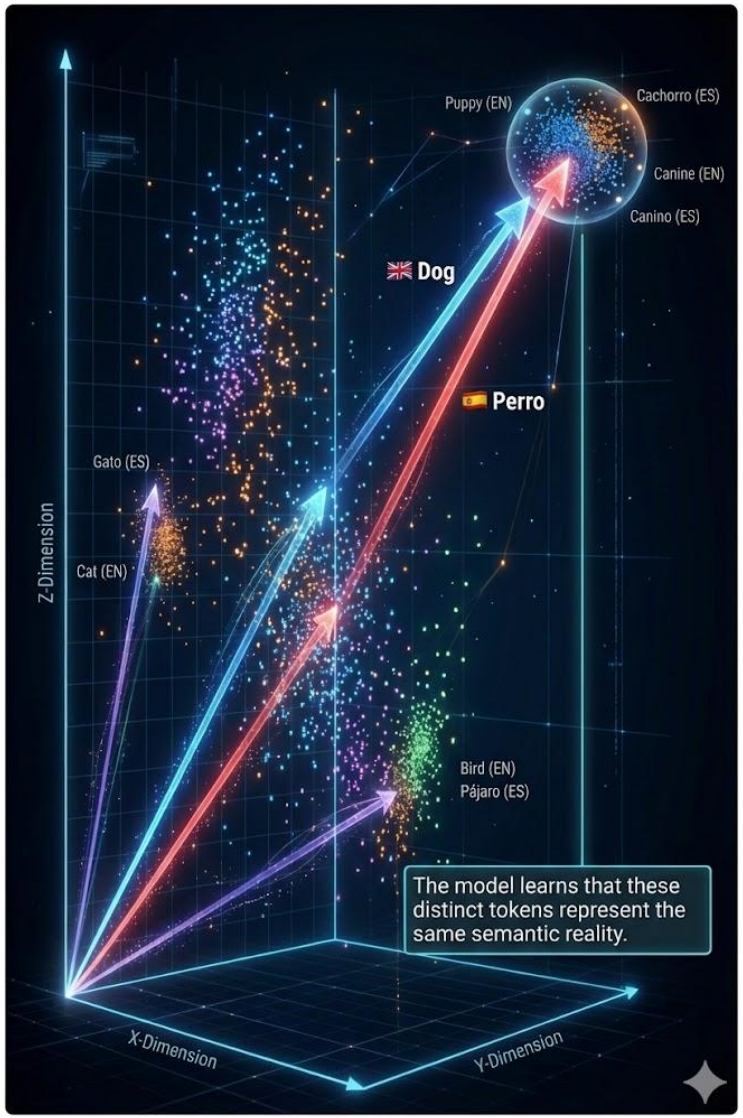
Why Multilingual Modeling Works

- Most languages share deep patterns: subjects, verbs, objects etc
- Subword-level tokenization allows shared representation of concepts
- Even unrelated scripts partially overlap in semantic space
- This is why a model trained on English + French often works on Spanish

© 2025 TekFrameworks

Shared Embedding Spaces

- **Unified Vocabulary**: Instead of separate vocabs, we use a massive shared vocabulary (often 250k + tokens)
- **Alignment**: During pre-training (Masked Language Modeling), the model sees parallel text (or enough mixed text) to align the vector spaces of both languages.
- **Result**: A “semantic universal language” inside the model’s weights



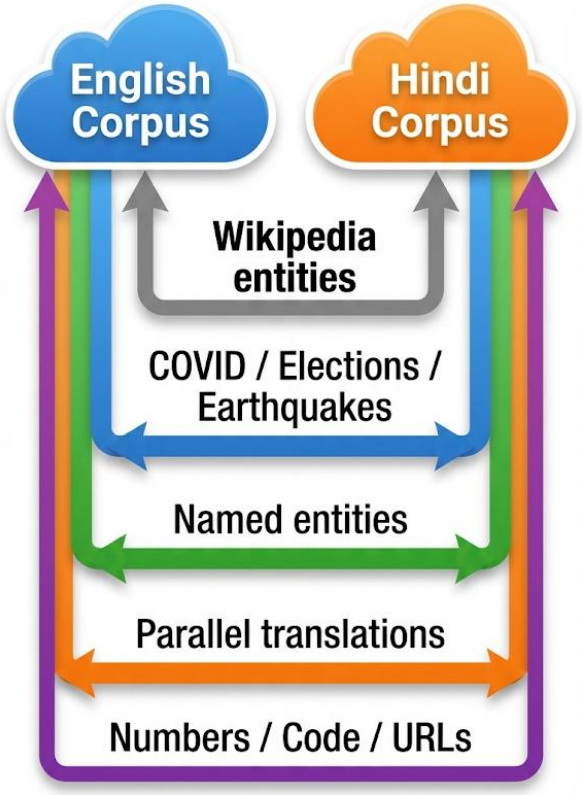
© 2025 TekFrameworks

Where Does Cross-Lingual Alignment Actually Come From?

- Random English + Hindi text alone does NOT align languages

- Alignment emerges only because training data contains shared anchors:
  - Wikipedia is available in 100s of languages
  - Named entities (Modi, COVID-19, iPhone, GDP, dates, URLs)
  - Global events reported in every language
  - Code snippets & technical content embedded inside local blogs

- These anchors stitch together separate language manifolds
- Without them, multilingual transfer would collapse.



Shared Anchors → Semantic Stitching

© 2025 TekFrameworks

```python
from transformers import AutoTokenizer

# Load a popular multilingual tokenizer (XLM-RoBERTa)
# Vocabulary size: ~250,000 tokens
tokenizer = AutoTokenizer.from_pretrained('xlm-roberta-base')

print(f"Vocab Size: {tokenizer.vocab_size}")

# 1. THE SHARED ANCHOR EFFECT
# Notice how 'Paris' gets the SAME Token ID (5676), regardless of the sentence lang
# This acts as the mathematical 'anchor' we discussed.
en_text = "I love Paris."
fr_text = "J'adore Paris."

print(f"\nEnglish: {tokenizer.encode(en_text)}")
print(f"French:  {tokenizer.encode(fr_text)}")
# Output English: [0, 87, 5155, 5676, 5, 2]  <- See 5676
# Output French:  [0, 688, 705, 30720, 5676, 5, 2] <- See 5676
```

```python
# 2. SCRIPT AGNOSTICISM
# The tokenizer handles Hindi, Chinese, and English using the same lookup table.
# It doesn't switch modes.
mixed_text = "Hello नमस्ते 你好"
tokens = tokenizer.tokenize(mixed_text)
ids = tokenizer.encode(mixed_text)

print(f"\nOriginal: {mixed_text}")
print(f"Tokens:   {tokens}")
print(f"IDs:      {ids}")
```

© 2025 TekFrameworks

Tokenization at Scale: Handling the script barrier

- Byte-pair Encoding (BPE) & Sentence Piece: The standard for multilingual models
- Scripts with shared roots → more shared tokens (Latin, Cyrillic)
- For unrelated scripts, break into subword units and share morphemes.
- Large vocab (~ 250k tokens), supports dozens of languages efficiently.
- Caution: If you domain uses a niche language(eg: Tulu), stock tokenizers might fragment words too aggressively, destroying semantic meaning

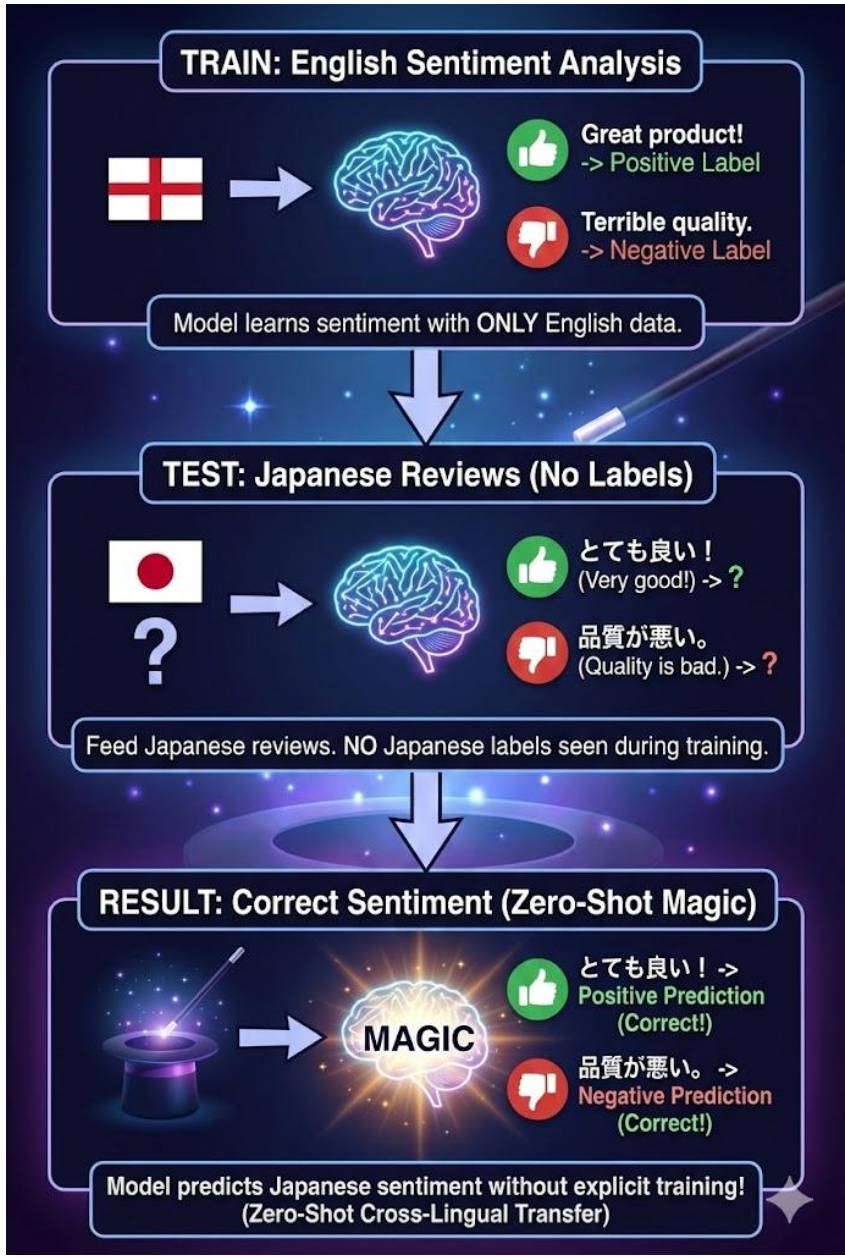
© 2025 TekFrameworks

Zero-Shot Cross-Lingual Transfer

- Defintion: Fine-tuning a model on a downstream task in a source language (usually English) and evaluating it on a target language without seeing any labelled data in the target language

- Why this matters for ROI: You don't need to pay for labelling data in 50 languages. Label once (English), deploy everywhere

- Limitations:
  - Performance drops on "culturally specific" nuances (idioms, sarcasm).
  - As you add more languages to a fixed-model, performance on all languages eventually drops



© 2025 TekFrameworks

The Hero Models

- Encoders (Understanding):
  - mBert: The grandfather. Good for classification
  - XLM-R (RoBERTa): Trained on 100 languages (CommonCrawl). The standard for non-generative tasks
- Encoder-Decoder (Translation):
  - NLLB (No Language Left Behind): Meta's models capable of 200+ languages
- Decoders (Generation):
  - BLOOM: The first massive open multilingual LLM.
  - GPT-4/Llama 3: Implicitly multilingual due to massive websrape

© 2025 TekFrameworks

From Languages to Images…

- We have spent time understanding text generation..
- Lets spend now try understand the image generation process..

© 2025 TekFrameworks

A Brief history of image generation models

© 2025 TekFrameworks

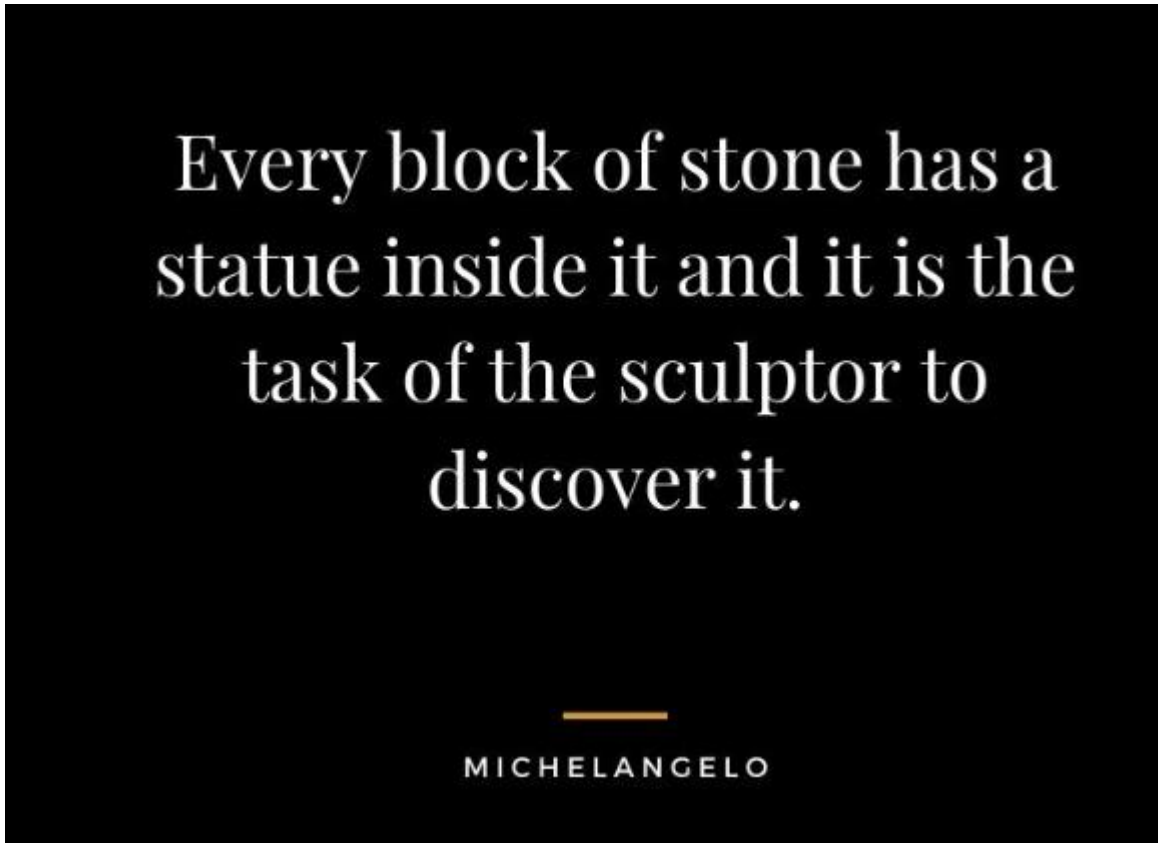
The core idea

You ask an experienced artist to help clean up an image painted by someone else – but has been corrupted by noise.

Tell them that this is a “Painting of an alien playing a guitar in the style of H.R. Giger”. Ask them to use this info and clean up.

From: https://mccormickml.com/2022/12/21/how-stable-diffusion-works/

© 2025 TekFrameworks



Like a Michelangelo, Diffusion models “sees” the painting in a noisy palette and goes ahead removing the noise step-by-step until the full picture is visible.

© 2025 TekFrameworks

"Removing Noise" over 50 steps

1 20 30 40 50

From: https://mccormickml.com/2022/12/21/how-stable-diffusion-works/

© 2025 TekFrameworks

Diffusion Model: Behind the scenes

- Diffusion doesn’t memorize pictures - it memorizes the *process of cleaning them*.
- Diffusion models have two phases: Forward phase where noise is added and the reverse phase where noise is removed
- During training the model learns to predict *just the noise* added at each step. Much like a restorer who first damages a photo on purpose and then masters the art of repairing it.
- Each step slightly cleans the image, guided by what the model *thinks* should be there

© 2025 TekFrameworks

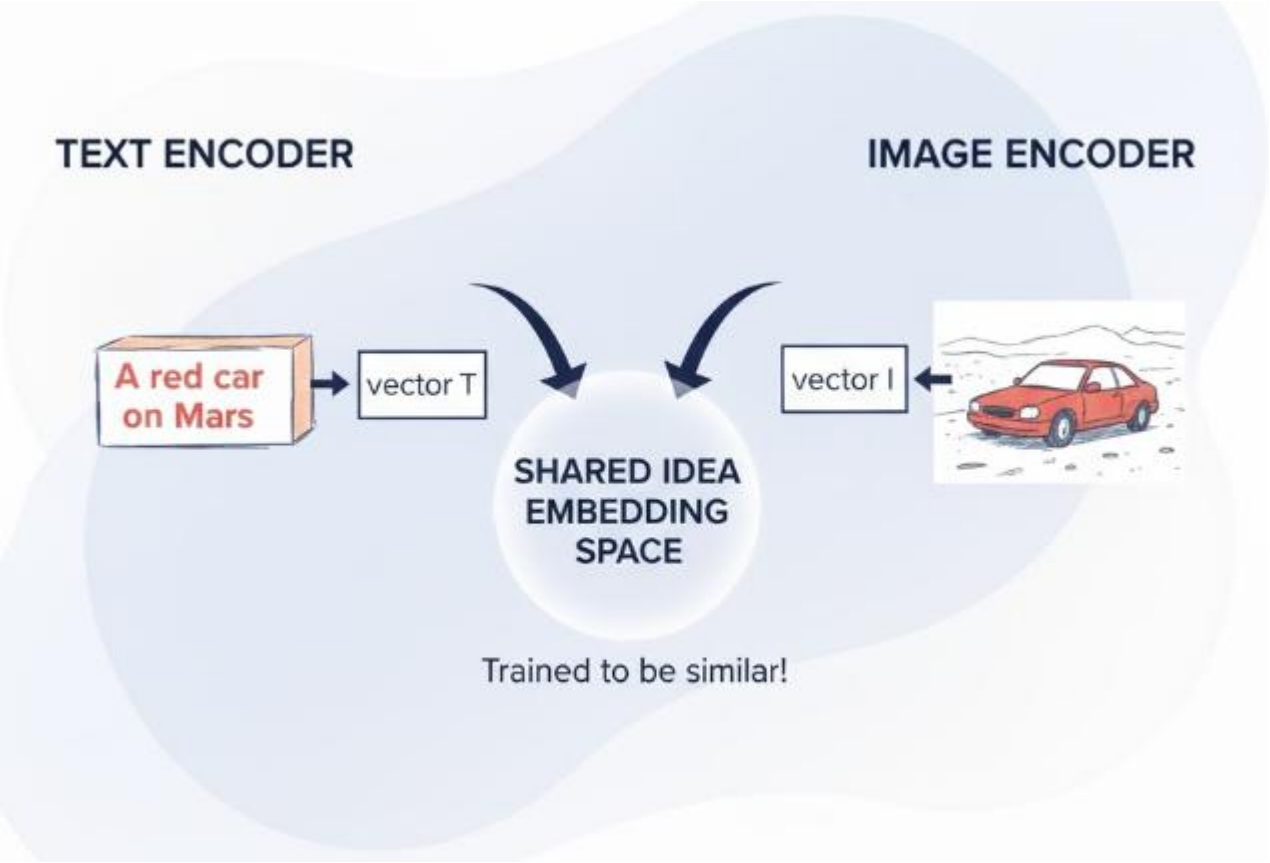
- How does it relate the image to the caption provided?

- That’s where CLIP comes in…

© 2025 TekFrameworks

CLIP: Contrastive Language-Image Pre-Training

- Developed by OpenAI in 2021 using 400M image-caption pairs
- Text encoder + Image Encoder trained together
- Goal: Map both words and pictures into the same contextual space



CLIP's shared embedding space is what allows a single vector DB to store and retrieve both text and images.

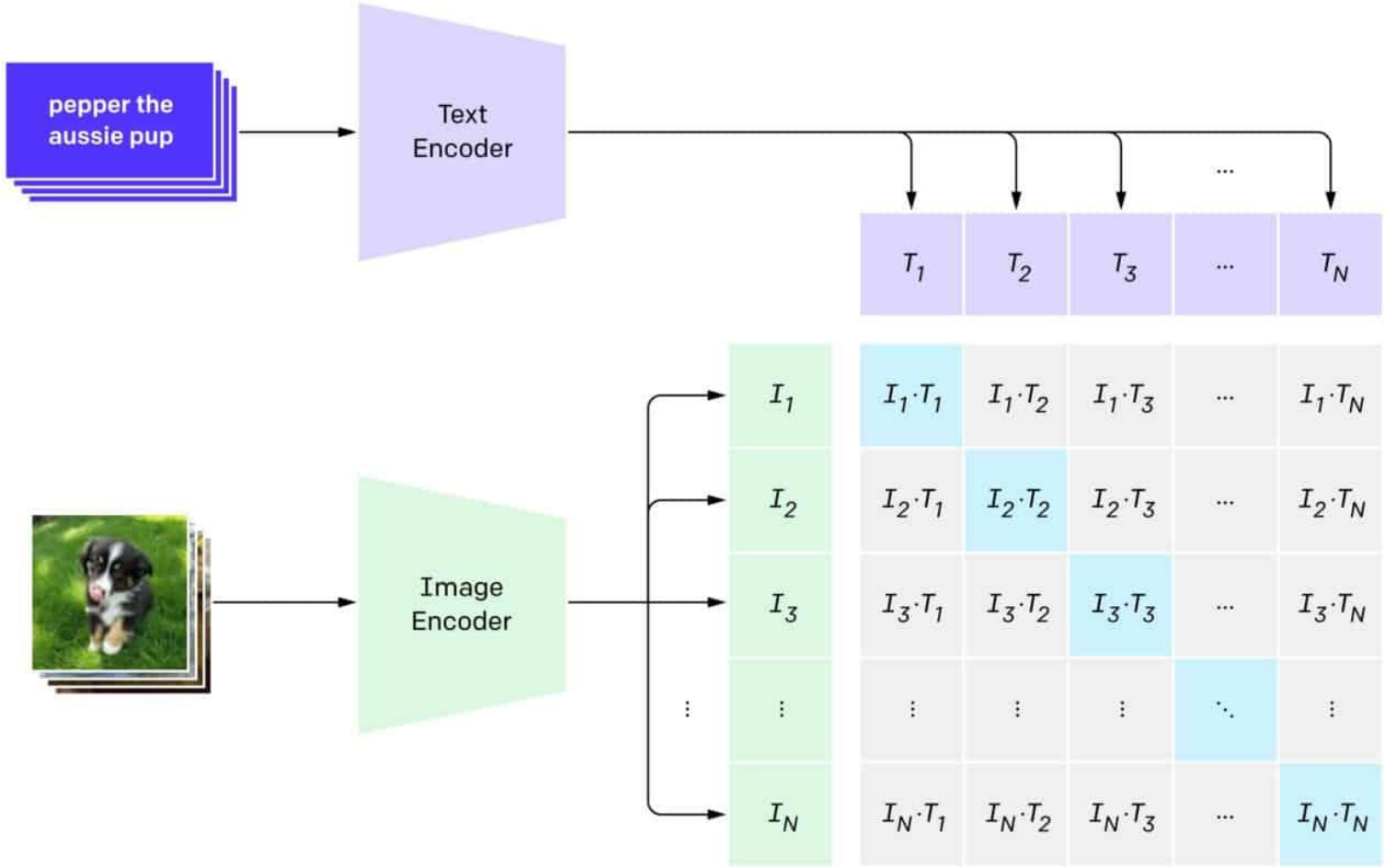
© 2025 TekFrameworks

CLIP training

From: https://www.youtube.com/watch?v=iv-5mZ_9CPY

© 2025 TekFrameworks

CLIP training



From: https://viso.ai/deep-learning/clip-machine-learning/

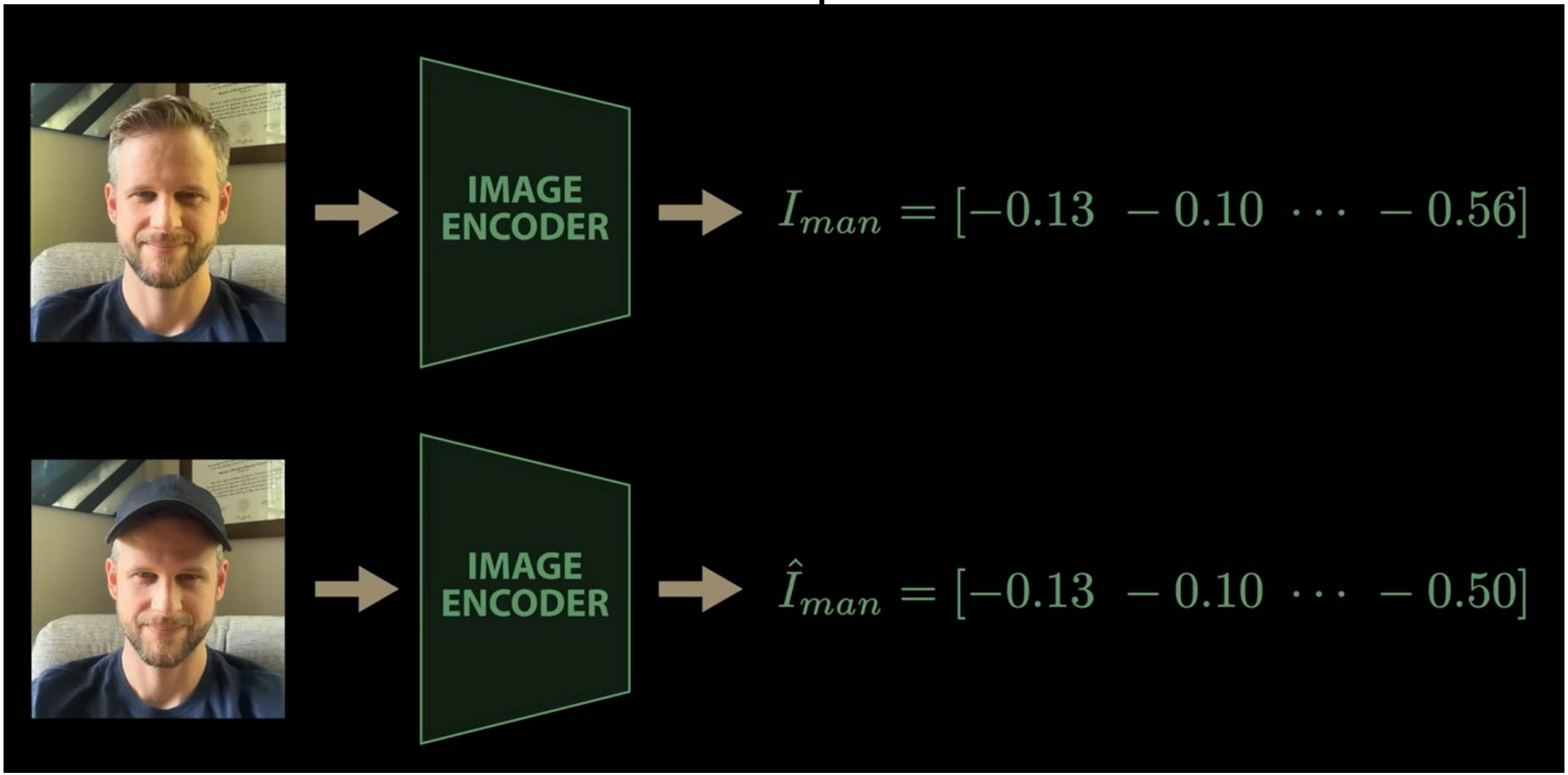
© 2025 TekFrameworks

CLIP

- Learns by *contrasting* correct and incorrect pairs.
- Matching pairs → closer in space.
- Non-matching pairs → pushed apart.
- Outcome: related text & image vectors point in same direction.

© 2025 TekFrameworks

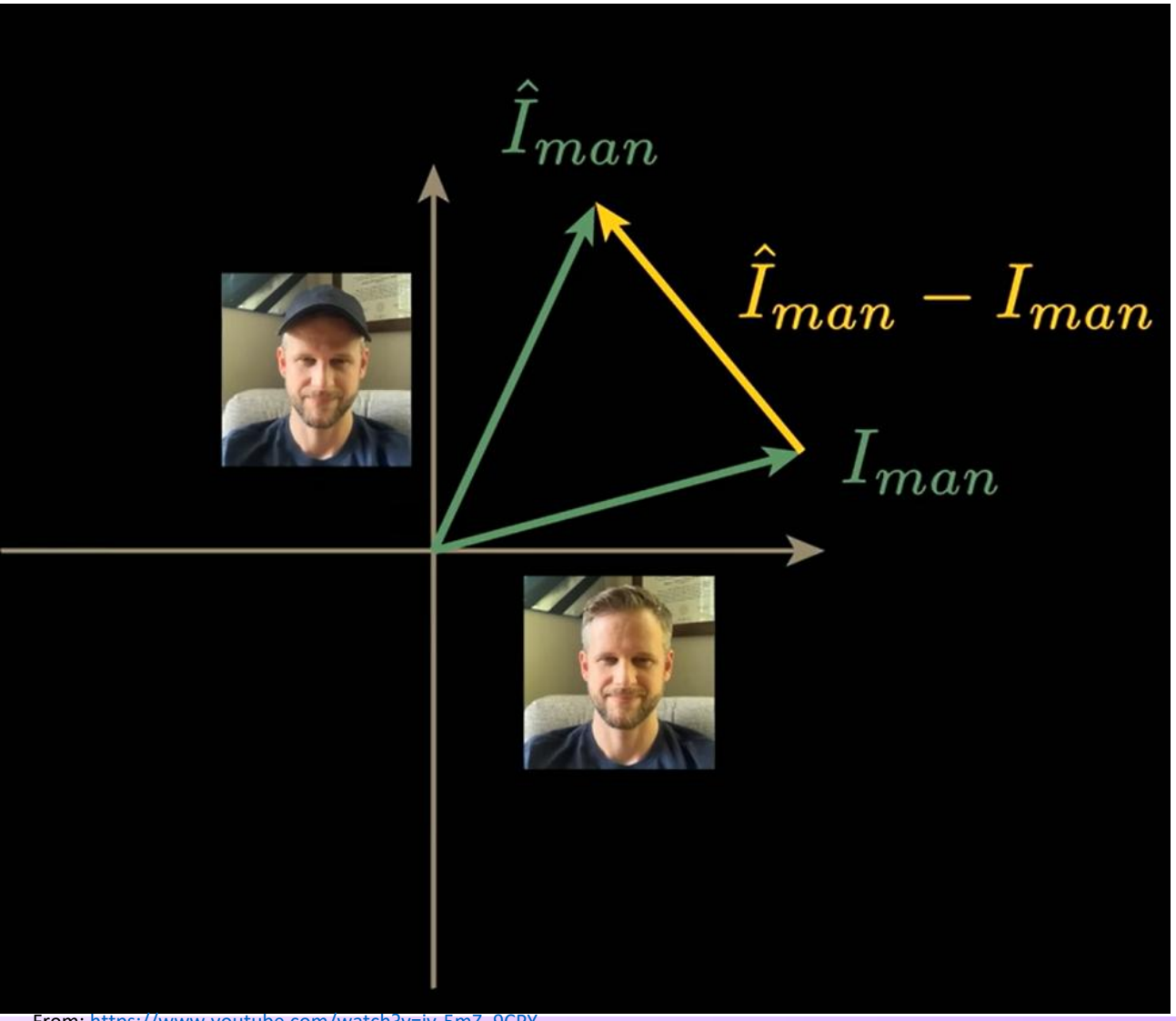
CLIP learns semantic representatiion



From: https://www.youtube.com/watch?v=iv-5mZ_9CPY

© 2025 TekFrameworks

CLIP learns semantic representatiion



TOP MATCHES

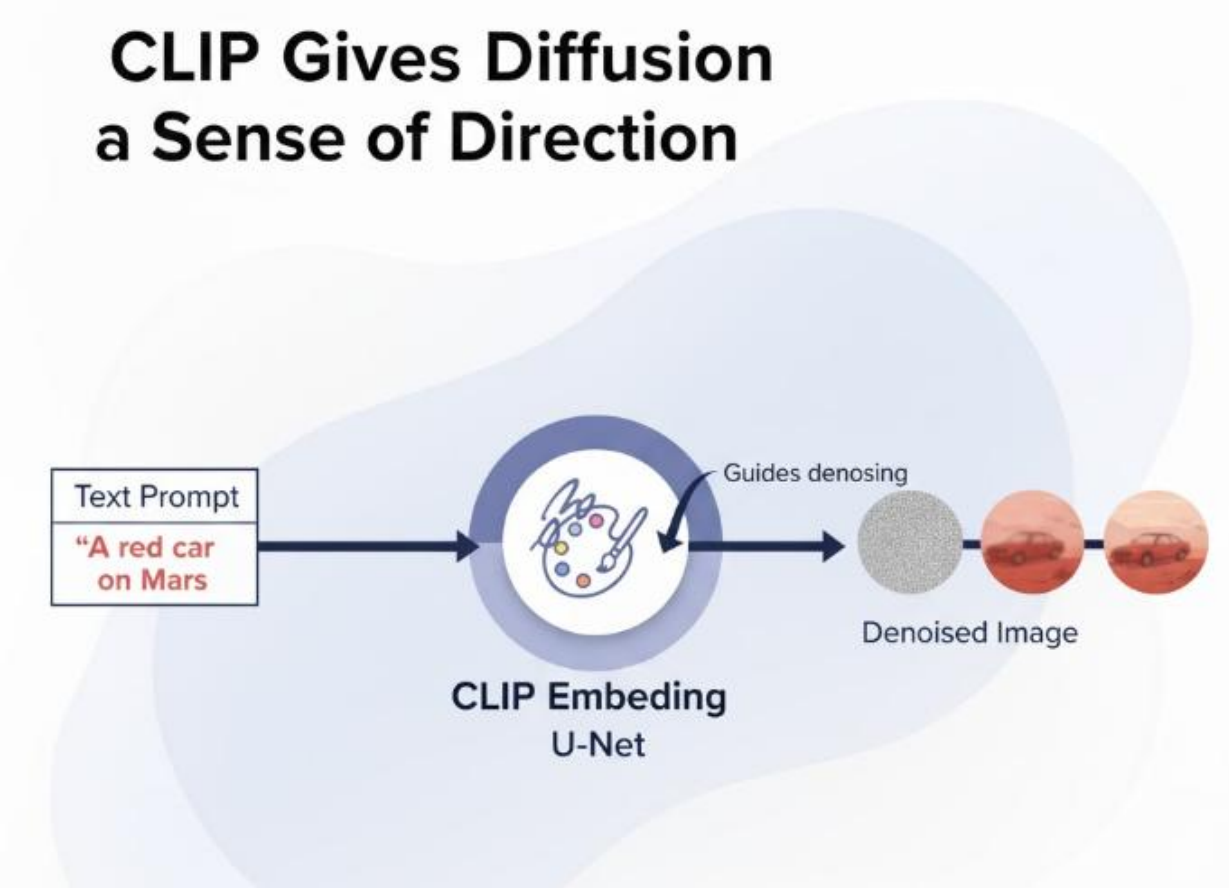
hat	0.165
cap	0.113
helmet	0.106
angry	0.061
exercising	0.06

From: https://www.youtube.com/watch?v=iv-5mZ_9CPY

© 2025 TekFrameworks

Conditioning the Process with CLIP

- Text prompt → encoded as a CLIP vector in shared idea space.
- This vector conditions the U-Net at each denoising step.
- The model removes noise not randomly, but toward that concept.



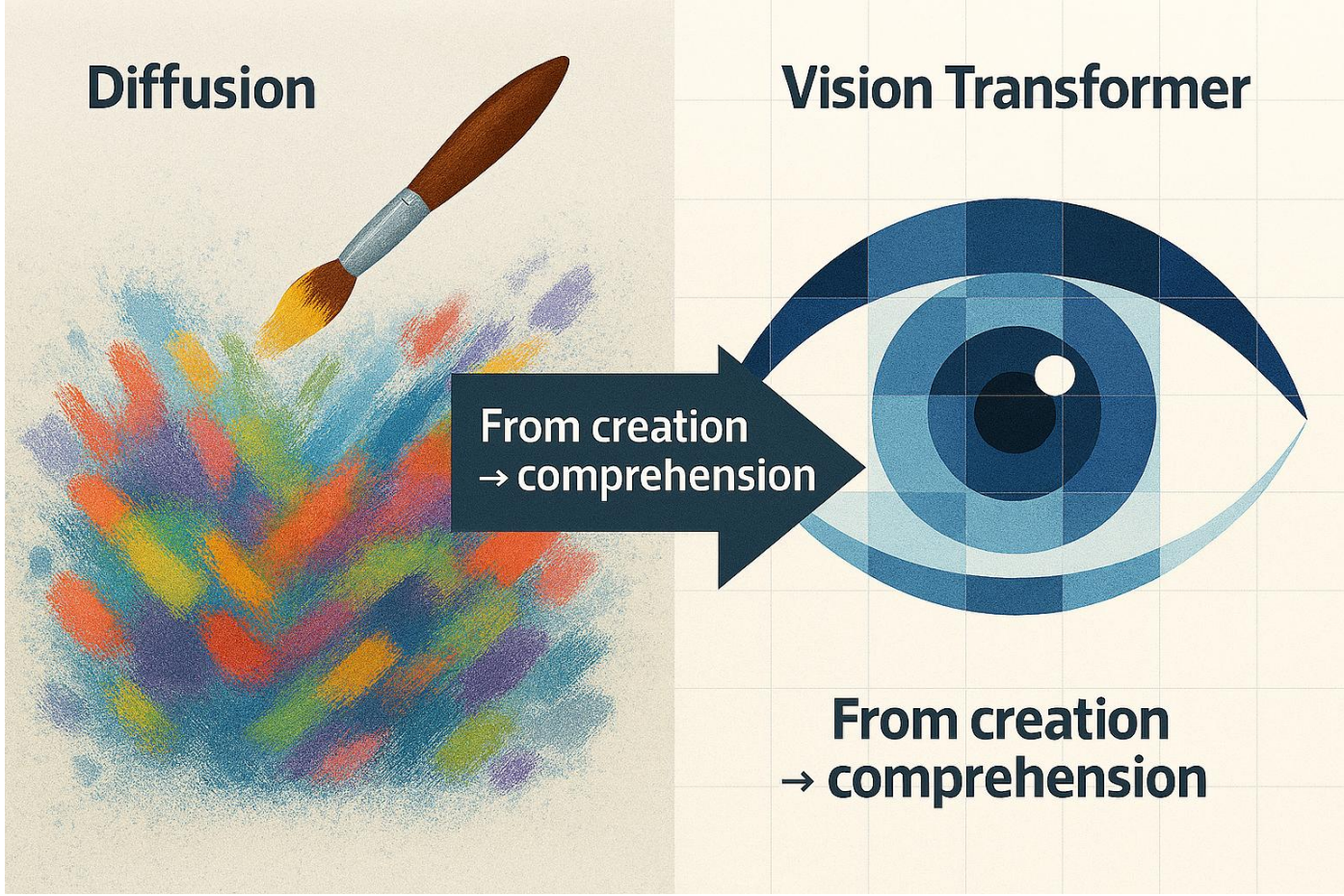
© 2025 TekFrameworks

Vision transformers

© 2025 TekFrameworks

From Pixels to Perception

- Now that we can paint, can we use the same mechanism that allowed us to understand text to understand pictures as well?
- Can transformers mechanism be used to understand images?



© 2025 TekFrameworks

ViT: The Tokenization Trick Applied to Images

- **The core idea (you already know this):**
- In BERT: words → tokens → embeddings → self-attention
- In ViT: image patches → tokens → embeddings → self-attention

- **The only new idea:**
- Chop the image into 16x16 pixel squares. Each square becomes a token.
- Add position embeddings (so the model knows where each patch came from).
- Feed the full sequence into a standard transformer encoder.

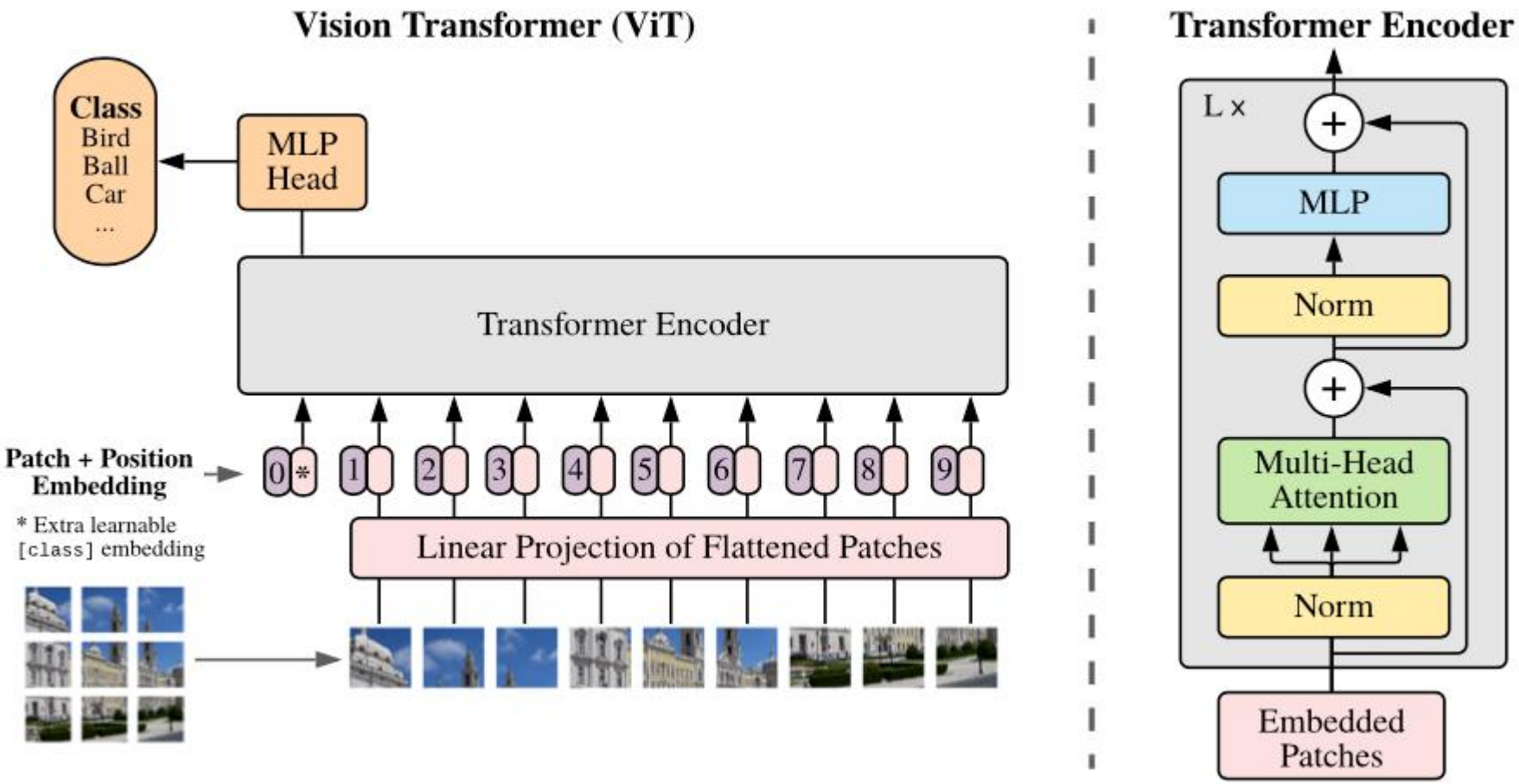
- **Key benefit:**
- Every patch can attend to every other patch — no locality constraint.
- A number in the top-left table can relate to a figure at the bottom-right.

© 2025 TekFrameworks

Why ViT Matters: It Speaks Transformer

- **The real question: why does the image encoder architecture matter?**
- ViT patch embeddings are transformer-compatible — just vectors in a sequence.
- Concatenate image patch tokens + text tokens into ONE sequence.
- Feed that single sequence into the same transformer decoder.
- Attention layers do not know (or care) which tokens came from the image.
- **This is exactly what GPT-4o and Gemini do.**
- Image + question → tokens in one sequence → one forward pass — native multimodality.
- ***ViT is not primarily better at seeing — it is better at talking to other transformers.***

© 2025 TekFrameworks

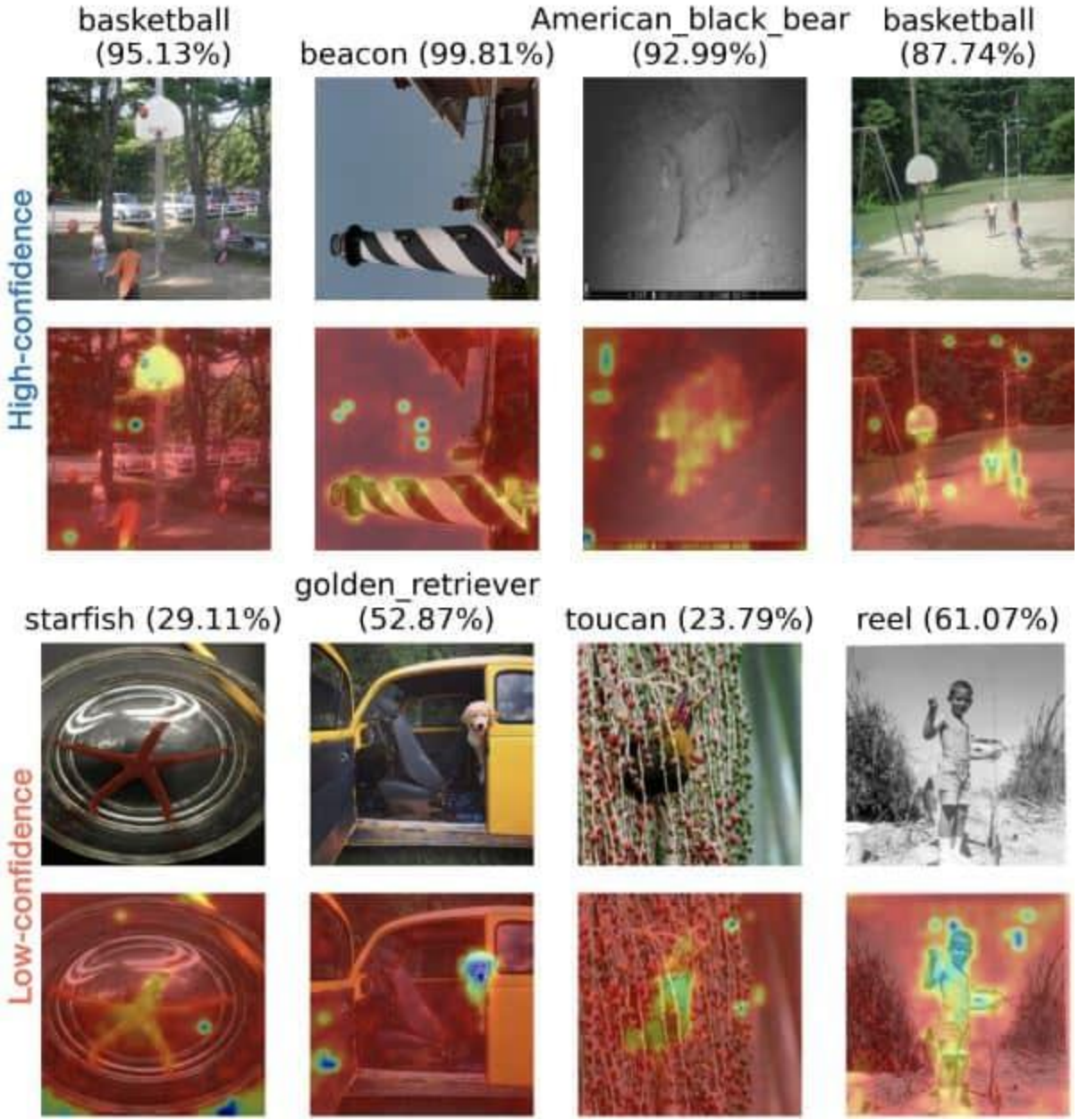


CNNs were like looking through a keyhole - sharp locally but missing the whole scene.

Vision Transformers read the image like text (patch by patch) and use attention to connect the story.

From: https://viso.ai/deep-learning/vision-transformer-vit/

© 2025 TekFrameworks



Visualization of attention maps in VIT from: https://viso.ai/deep-learning/vision-transformer-vit/

© 2025 TekFrameworks

Image Captioning & Cross-Modal Understanding

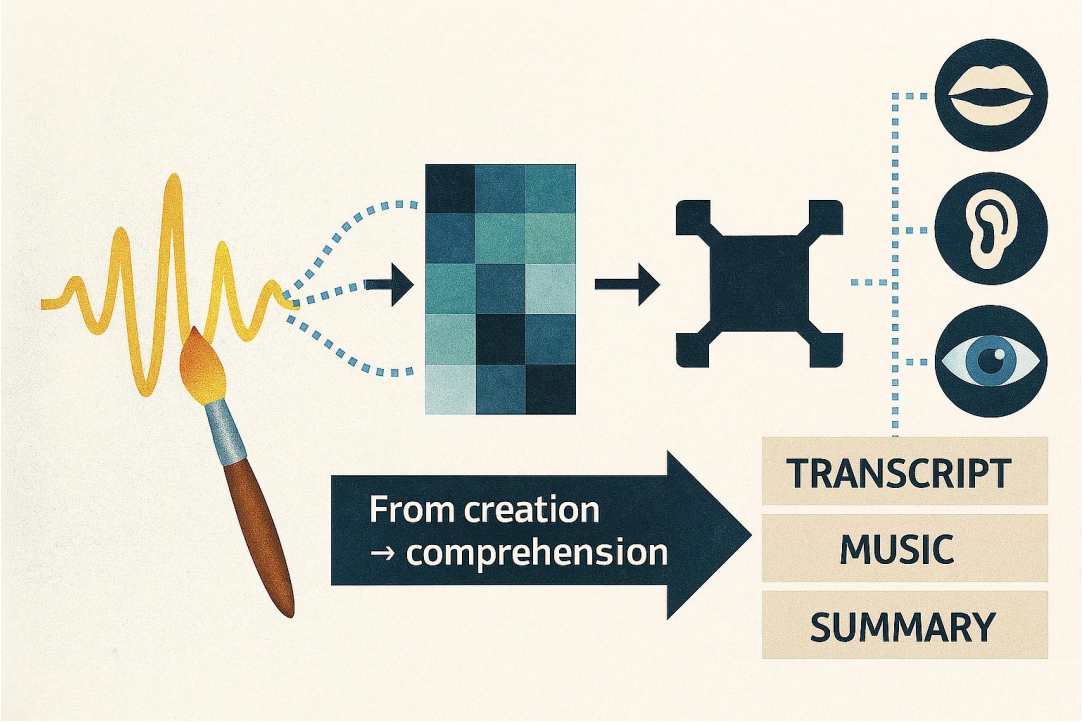
- Combine a Vision Transformer encoder + Language decoder → models that describe images in words.
- Early examples: BLIP, Flamingo (Google DeepMind).
- Used for search, accessibility, creative briefing, and context-aware agents.

- Vision and Language started sharing a common Transformer brain

From: https://www.kaggle.com/code/burhanuddinlatsaheb/image-captioning-vit-gpt2

Audio Transformers – Giving AI Ears

- Audio is just another sequence — a waveform or spectrogram of tokens.
- Models like Whisper, AudioLM, MusicGen apply Transformers to sound.
- This completes the triad of modalities: Text, Image & Audio

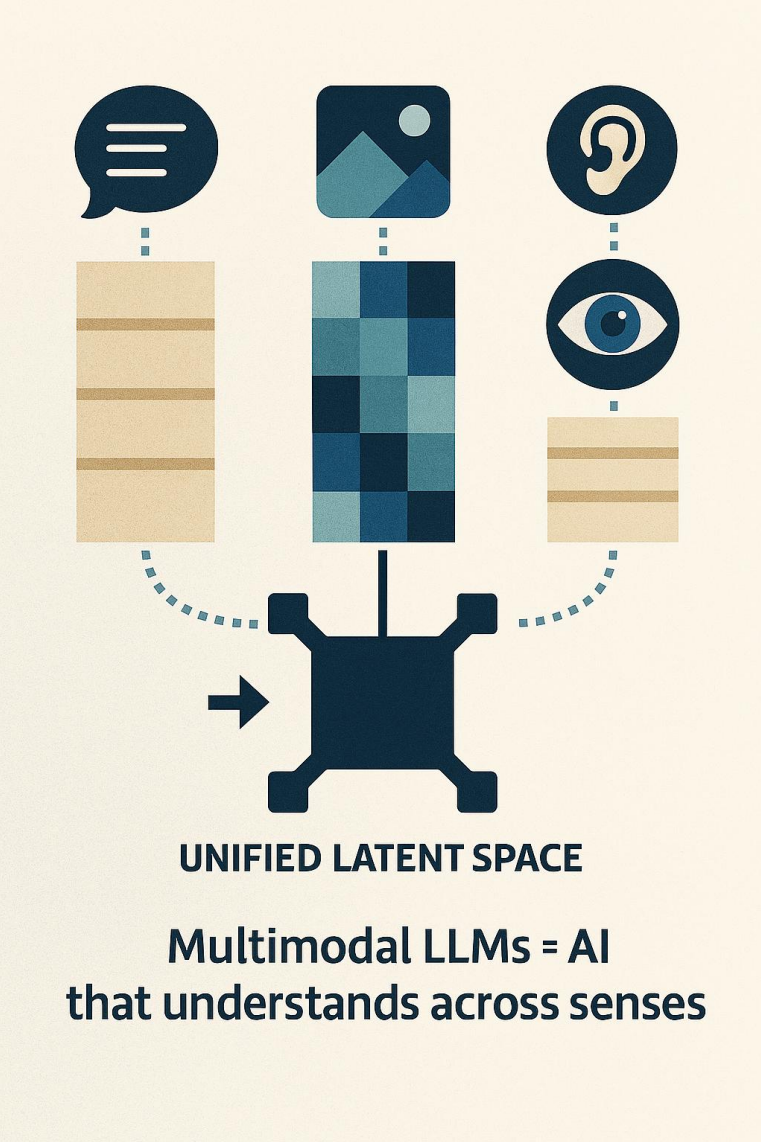


Transformers turned out to be universal; if you can tokenize it, they can learn it.

© 2025 TekFrameworks

The Rise of Multimodal LLMs

- Modern LLMs (GPT-4o, Gemini, Claude 3, LLaVA) integrate multiple encoders into one shared transformer
- They reason across modalities (“*How many people are in this photo?*”)
- Enables true contextual AI agents which can read docs, see dashboards and listen to audio



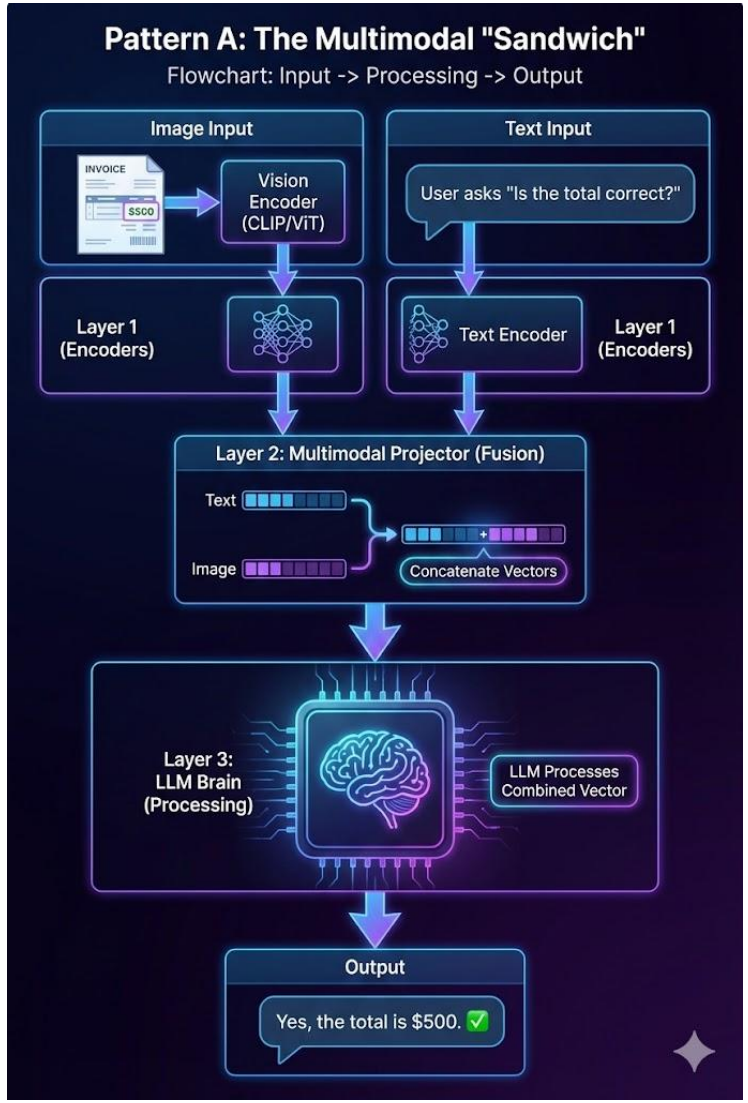
© 2025 TekFrameworks

The Enterprise Architecture

© 2025 TekFrameworks

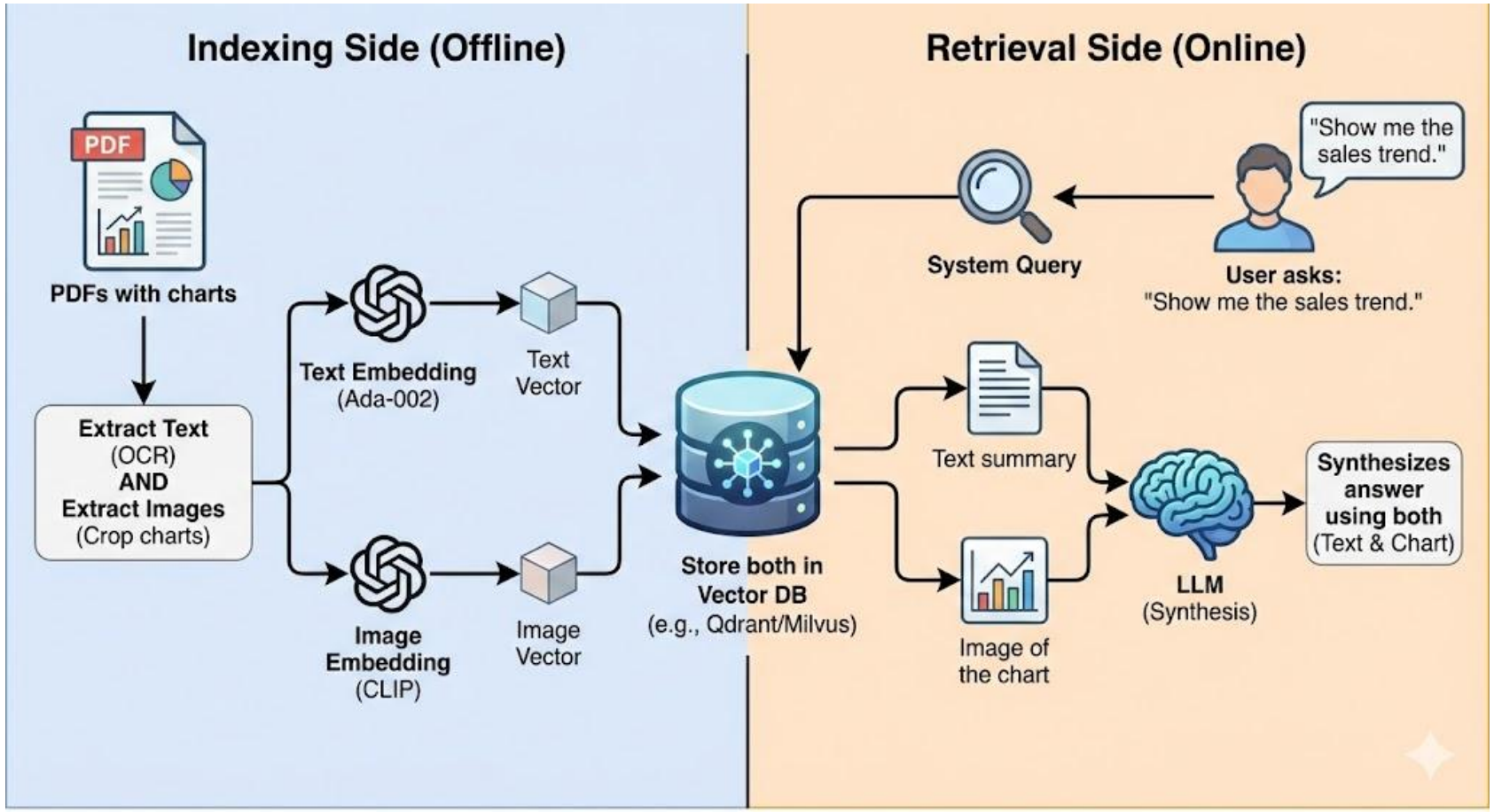
The Canonical Multimodal pipeline

- Layer 1: Independent Encoders (The senses)
  - Text & Images are processed by specialized pre-trained encoders
  - Raw pixels → high dimensional feature vectors
- Layer 2: The multimodal projector (The glue)
  - The visual vectors & text vectors are combined
- Layer 3: The LLM (The Brain)
  - LLM processes the combined sequence (image + text) in one pass
  - Cross-attention: the model links the text query to specific features in the image to answer questions



© 2025 TekFrameworks

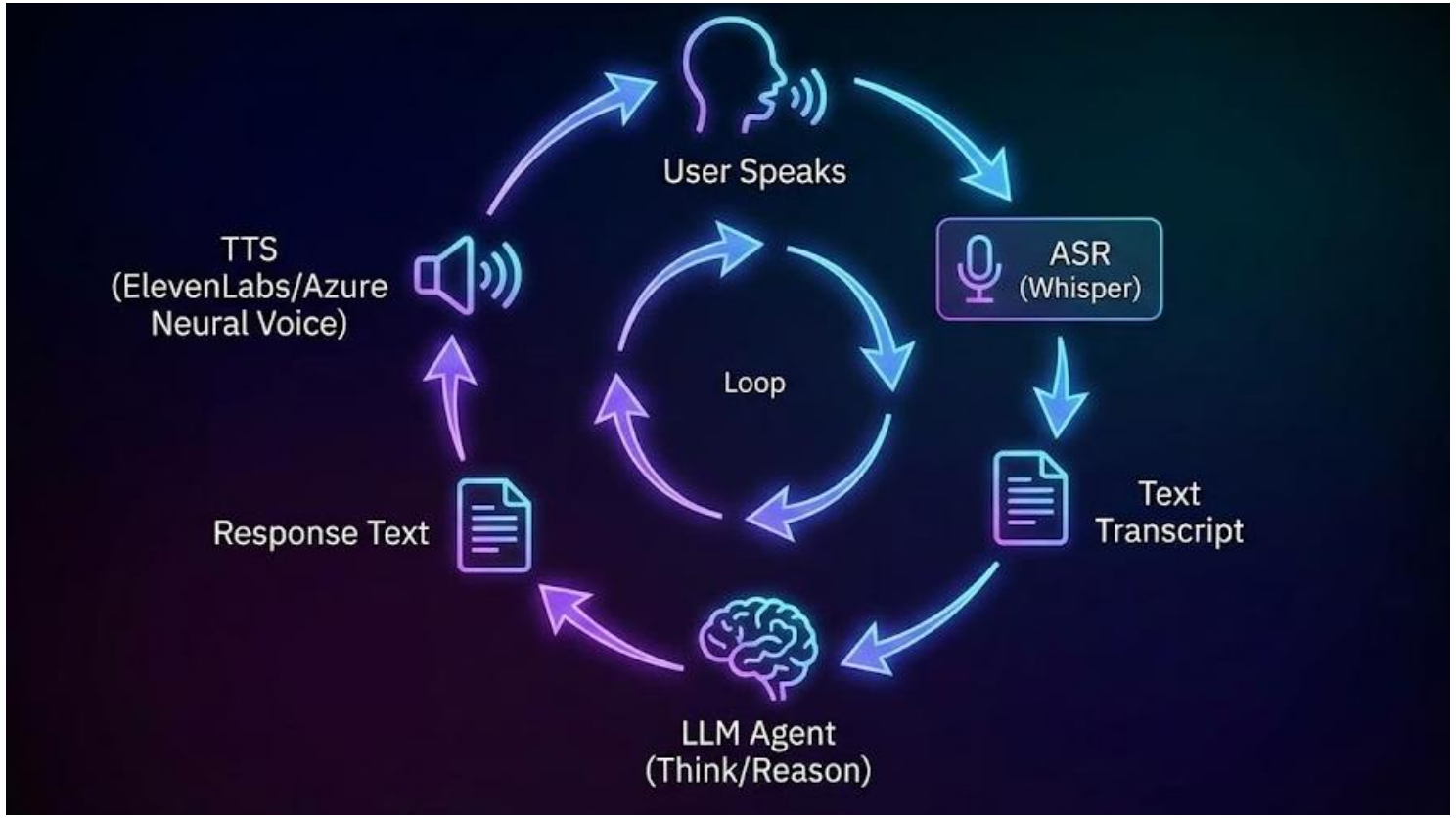
Pattern B: Multimodal RAG



CLIP embeddings make this possible, since image and text embeddings share the same vector space, one query retrieves from both.

© 2025 TekFrameworks

Pattern C: Voice first flow



- Engineering Constraints
  - Voice activity detection: Knowing when the user stopped speaking is the one of the hardest part.
  - Latency Budget: You have ~800ms total for it to feel “conversational”.
  - Streaming: You must stream the text from the LLM directly into the TTS engine chunk-by-chunk

© 2025 TekFrameworks

Metrics

- Text Metrics:
  - BLEU/ROUGE (looks for exact word matches)
  - BERTScore/LLM-as-a-judge (Semantic matches)

- Multimodal Metrics:
  - Accuracy: Did it answer the question about the image correctly?
  - Hallucination Rate: Model claims “There is a red car” when there is none.
  - Clip Score: How well does the generated caption align with the image?

- For generative images, human evaluation is still the gold standard.

© 2025 TekFrameworks

Summary: The Unified Model

- **Multilingual**: We solved language barriers by sharing concept embeddings, not just translating words
- **Multimodal**: We solve sensory barriers (vision/audio) by aligning them to the same text embedding space (CLIP)
- **Architecture**: Modern apps don't just generate text – they read documents, see charts and listen to voice using specialized encoders fused into a LLM.

- **Next** : Autonomous decision making with..

© 2025 TekFrameworks

Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekFrameworks.com | www.tekFrameworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2025 TekFrameworks